Tokenization Is More Than Compression

Craig W. Schmidt[†] **Varshini Reddy**[†] **Haoran Zhang**[†,‡] **Alec Alameddine**[†]
Omri Uzan[§] **Yuval Pinter**[§] **Chris Tanner**[†,¶]

[†]Kensho Technologies Cambridge, MA
[‡]Harvard Univ Cambridge, MA
[§]Dept of Computer Science Ben-Gurion Univ of the Negev Beer Sheva, Israel
[¶]MIT Cambridge, MA

{craig.schmidt,varshini.reddy,alec.alameddine,chris.tanner}@kensho.com
haoran_zhang@g.harvard.edu {omriuz@post,uvp@cs}.bgu.ac.il

Abstract

Tokenization is a foundational step in natural language processing (NLP) tasks, bridging raw text and language models. Existing tokenization approaches like Byte-Pair Encoding (BPE) originate from the field of data compression, and it has been suggested that the effectiveness of BPE stems from its ability to condense text into a relatively small number of tokens. We test the hypothesis that fewer tokens lead to better downstream performance by introducing PathPiece, a new tokenizer that segments a document's text into the minimum number of tokens for a given vocabulary. Through extensive experimentation we find this hypothesis not to be the case, casting doubt on the understanding of the reasons for effective tokenization. To examine which other factors play a role, we evaluate design decisions across all three phases of tokenization: pre-tokenization, vocabulary construction, and segmentation, offering new insights into the design of effective tokenizers. Specifically, we illustrate the importance of pre-tokenization and the benefits of using BPE to initialize vocabulary construction. We train 64 language models with varying tokenization, ranging in size from 350M to 2.4B parameters, all of which are made publicly available.

1 Introduction

Tokenization is an essential step in NLP that translates human-readable text into a sequence of distinct tokens that can be subsequently used by statistical models (Grefenstette, 1999). Recently, a growing number of studies have researched the effects of tokenization, both in an intrinsic manner and as it affects downstream model performance (Singh et al., 2019; Bostrom and Durrett, 2020; Hofmann et al., 2021, 2022; Limisiewicz et al., 2023; Zouhar et al., 2023b). To rigorously inspect the impact of tokenization, we consider tokenization as three distinct, sequential stages:

1. **Pre-tokenization:** an optional set of initial rules that restricts or enforces the creation of certain tokens (e.g., splitting a corpus on whitespace, thus preventing any tokens from containing whitespace).

2. **Vocabulary Construction:** the core algorithm that, given a text corpus $\mathcal{C}$ and desired vocabulary size m, constructs a vocabulary of tokens $t_k \in \mathcal{V}$, such that $|\mathcal{V}| = m$, while adhering to the pre-tokenization rules.

3. **Segmentation:** given a vocabulary $\mathcal{V}$ and a document d, segmentation determines how to split d into a series of K_d tokens $t_1, \ldots, t_k, \ldots, t_{K_d}$, with all $t_k \in \mathcal{V}$, such that the concatenation of the tokens strictly equals d. Given a corpus of documents $\mathcal{C}$, we will define the corpus token count (CTC) as the total number of tokens used in each segmentation, $\mathrm{CTC}(\mathcal{C}) = \sum_{d \in \mathcal{C}} K_d$.

   As an example, segmentation might decide to split the text `intractable` into "`int ract able`", "`in trac table`", "`in tractable`", or "`int r act able`".

   We will refer to this step as segmentation, although in other works it is also called "inference" or even "tokenization".

The widely used Byte-Pair Encoding (BPE) tokenizer (Sennrich et al., 2016) originated in the field of data compression (Gage, 1994). Gallé (2019) argues that it is effective because it compresses text to a short sequence of tokens. Goldman et al. (2024) varied the number of documents in the tokenizer training data for BPE, and found a correlation between CTC and downstream performance. To investigate the hypothesis that having fewer tokens necessarily leads to better downstream performance, we design a novel tokenizer, PATHPIECE, that, for a given document d and vocabulary $\mathcal{V}$, finds a segmentation with the minimum possible

Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 678–702
November 12-16, 2024 ©2024 Association for Computational Linguistics

K_d. The PATHPIECE vocabulary construction routine is a top-down procedure that heuristically minimizes CTC on a training corpus. PATHPIECE is ideal for studying the effect of CTC on downstream performance, as we can vary decisions at each tokenization stage.

We extend these experiments to the most commonly used tokenizers, focusing on how downstream task performance is impacted by the major stages of tokenization and vocabulary sizes. Toward this aim, we conducted experiments by training 64 language models (LMs): 54 LMs with 350M parameters; 6 LMs with 1.3B parameters; and 4 LMs with 2.4B parameters. We provide open-source, public access to PATHPIECE[1], and our trained vocabularies and LMs[2].

2 Preliminaries

Ali et al. (2024) and Goldman et al. (2024) examined the effect of tokenization on downstream performance of LLM tasks, reaching opposite conclusions on the importance of CTC. Zouhar et al. (2023a) also find that low token count alone does not necessarily improve performance. Mielke et al. (2021) give a survey of subword tokenization.

2.1 Pre-tokenization Methods

Pre-tokenization is a process of breaking text into chunks, which are then tokenized independently. A token is not allowed to cross these pre-tokenization boundaries. BPE, WordPiece, and Unigram all require new chunks to begin whenever a space is encountered. If a space appears in a chunk, it must be the first character; hence, we will call this "FirstSpace". Thus "␣New" is allowed but "New␣York" is not. Gow-Smith et al. (2022) examine treating spaces as individual tokens, which we will call "Space" pre-tokenization, while Jacobs and Pinter (2022) suggest marking spaces at the end of tokens, and Gow-Smith et al. (2024) propose dispensing them altogether in some settings. Llama (Touvron et al., 2023) popularized the idea of having each digit always be an individual token, which we call "Digit" pre-tokenization.

2.2 Vocabulary Construction

We focus on byte-level, lossless subword tokenization. Subword tokenization algorithms split text into word and subword units based on their frequency and co-occurrence patterns from their "training" data, effectively capturing morphological and semantic nuances in the tokenization process (Mikolov et al., 2011).

We analyze BPE, WordPiece, and Unigram as baseline subword tokenizers, using the implementations from HuggingFace[3] with `ByteLevel` pre-tokenization enabled. We additionally study SaGe, a context-sensitive subword tokenizer, using version 2.0.[4]

Byte-Pair Encoding Sennrich et al. (2016) introduced Byte-Pair Encoding (BPE), a bottom-up method where the vocabulary construction starts with single bytes as tokens. It then merges the most commonly occurring pair of adjacent tokens in a training corpus into a single new token in the vocabulary. This process repeats until the desired vocabulary size is reached. Issues with BPE and analyses of its properties are discussed in Bostrom and Durrett (2020); Klein and Tsarfaty (2020); Gutierrez-Vasques et al. (2021); Yehezkel and Pinter (2023); Saleva and Lignos (2023); Liang et al. (2023); Lian et al. (2024); Chizhov et al. (2024); Bauwens and Delobelle (2024). Zouhar et al. (2023b) build an "exact" algorithm which optimizes compression for BPE-constructed vocabularies.

WordPiece WordPiece is similar to BPE, except that it uses Pointwise Mutual Information (PMI) (Bouma, 2009) as the criteria to identify candidates to merge, rather than a count (Wu et al., 2016; Schuster and Nakajima, 2012). PMI prioritizes merging pairs that occur together more frequently than expected, relative to the individual token frequencies.

Unigram Language Model Unigram works in a top-down manner, starting from a large initial vocabulary and progressively pruning groups of tokens that induce the minimum likelihood decrease of the corpus (Kudo, 2018). This selects tokens to maximize the likelihood of the corpus, according to a simple unigram language model.

SaGe Yehezkel and Pinter (2023) proposed SaGe, a subword tokenization algorithm incorporating contextual information into an ablation loss via a skipgram objective. SaGe also operates top-down, pruning from an initial vocabulary to a desired size.

[1] https://github.com/kensho-technologies/pathpiece

[2] https://github.com/kensho-technologies/timtc_vocabs_models

[3] https://github.com/huggingface/tokenizers

[4] https://github.com/MeLeLBGU/SaGe

2.3 Segmentation Methods

Given a tokenizer and a vocabulary of tokens, segmentation converts text into a series of tokens. We included all 256 single-byte tokens in the vocabulary of all our experiments, ensuring any text can be segmented without out-of-vocabulary issues.

Certain segmentation methods are tightly coupled to the vocabulary construction step, such as merge rules for BPE or the maximum likelihood approach for Unigram. Others, such as the WordPiece approach of greedily taking the longest prefix token in the vocabulary at each point, can be applied to any vocabulary; indeed, there is no guarantee that a vocabulary will perform best downstream with the segmentation method used to train it (Uzan et al., 2024). Additional segmentation schemes include Dynamic Programming BPE (He et al., 2020), BPE-Dropout (Provilkov et al., 2020), and FLOTA (Hofmann et al., 2022).

3 PATHPIECE

Several efforts over the last few years (Gallé, 2019; Zouhar et al., 2023a, *inter alia*) have suggested that the empirical advantage of BPE as a tokenizer in many NLP applications, despite its unawareness of language structure, can be traced to its superior compression abilities, providing models with overall shorter sequences during learning and inference. Inspired by this claim we introduce PATHPIECE, a lossless subword tokenizer that, given a vocabulary $\mathcal{V}$ and document d, produces a segmentation minimizing the total number of tokens needed to split d. We additionally provide a vocabulary construction procedure that, using this segmentation, attempts to find a $\mathcal{V}$ minimizing the corpus token count (CTC).[5] PATHPIECE provides an ideal testing laboratory for the compression hypothesis by virtue of its maximally efficient segmentation.

3.1 Segmentation

PATHPIECE requires that all single-byte tokens are included in vocabulary $\mathcal{V}$ to run correctly. PATHPIECE works by finding a shortest path through a directed acyclic graph (DAG), where each byte i of training data forms a node in the graph, and two nodes j and i contain a directed edge if the byte segment $[j, i]$ is a token in $\mathcal{V}$. We describe PATHPIECE segmentation in Algorithm 1, where L is a limit on the maximum width of a token in bytes, which we set to 16. It has a complexity of $O(nL)$, which follows directly from the two nested `for`-loops. For each byte i in d, it computes the shortest path length $pl[i]$ in tokens up to and including byte i, and the width $wid[i]$ of a token with that shortest path length. In choosing $wid[i]$, ties between multiple tokens with the same shortest path length $pl[i]$ can be broken randomly, or the one with the longest $wid[i]$ can be chosen, as shown here.[6] Then, a backward pass constructs the shortest possible segmentation from the $wid[i]$ values computed in the forward pass.

Algorithm 1 PATHPIECE segmentation.

```
procedure PathPiece(d, V, L)
    n ← len(d)                                   ▷ document length
    pl[1 : n] ← ∞                                ▷ shortest path length
    wid[1 : n] ← 0                               ▷ shortest path tok width
    for e ← 1, n do                              ▷ token end
        for w ← 1, L do                          ▷ token width
            s ← e − w + 1                        ▷ token start
            if s ≥ 1 then                        ▷ s in range
                if d[s : e] ∈ V then
                    if s = 1 then                ▷ 1 tok path
                        pl[e] ← 1
                        wid[e] ← w
                    else
                        nl ← pl[s − 1] + 1
                        if nl ≤ pl[e] then
                            pl[e] ← nl
                            wid[e] ← w
    T ← []                                       ▷ output token list
    e ← n                                        ▷ start at end of d
    while e ≥ 1 do
        s ← e − wid[e] + 1                       ▷ token start
        T.append(d[s : e])                       ▷ append token
        e ← e − wid[e]                           ▷ back up a token
    return reversed(T)                           ▷ reverse order
```

3.2 Vocabulary Construction

PATHPIECE's vocabulary is built in a top-down manner, attempting to minimize the corpus token count (CTC), by starting from a large initial vocabulary $\mathcal{V}_0$ and iteratively omitting batches of tokens. The $\mathcal{V}_0$ may be initialized from the most frequently occurring byte n-grams in the corpus, or from a large vocabulary trained by BPE or Unigram. We enforce that all single-byte tokens remain in the vocabulary and that all tokens are L bytes or shorter.

For a PATHPIECE segmentation $t_1, \ldots, t_{K_d}$ of a document d in the training corpus $\mathcal{C}$, we would like to know the increase in the overall length of the segmentation K_d after omitting each token t from our vocabulary and then recomputing the segmen-

[5] An extended description is given in Appendix A.

[6] Random tie-breaking, which can be viewed as a form of subword regularization, is presented in Appendix A. Some motivation for selecting the longest token is due to the success of FLOTA (Hofmann et al., 2022).

tation. Tokens with a low overall increase are good candidates to remove from the vocabulary.

To avoid the very expensive $O(nL|\mathcal{V}|)$ computation of each segmentation from scratch, we make a simplifying assumption that allows us to compute these increases more efficiently: we omit a specific token t_k, for $k \in [1, \ldots, K_d]$ in the segmentation of a particular document d, and compute the minimum increase $MI_{kd} \geq 0$ in the total tokens K_d from not having that token t_k in the segmentation of d. We then aggregate these token count increases MI_{kd} for each token $t \in \mathcal{V}$. We can compute the MI_{kd} without actually re-segmenting any documents, by reusing the shortest path information computed by Algorithm 1 during segmentation.

Any segmentation not containing t_k must either contain a token boundary somewhere inside of t_k breaking it in two, or it must contain a token that entirely contains t_k as a superset. We enumerate all occurrences for these two cases, and we find the minimum increase MI_{kd} among them. Let t_k start at index s and end at index e, inclusive. Path length $pl[j]$ represents the number of tokens required for the shortest path up to and including byte j. We also run Algorithm 1 backwards on d, computing a similar vector of backwards path lengths $bpl[j]$, representing the number of tokens on a path from the end of the data up to and including byte j. The minimum length of a segmentation with a token boundary after byte j is thus:

$$K^b_j = pl[j] + bpl[j+1]. \tag{1}$$

We have added an extra constraint on the shortest path, that there is a break at j, so clearly $K^b_j \geq K_d$. The minimum increase for the case of having a token boundary within t_k is thus:

$$MI^b_{kd} = \min_{j=s,\ldots,e-1} K^b_j - K_d. \tag{2}$$

The minimum increase from omitting t_k could also be from a segmentation containing a strict superset of t_k. Let this superset token be t'_k, with start s' and end e' inclusive. To be a strict superset entirely containing t_k, then either $s' < s$ and $e' \geq e$, or $s' \leq s$ and $e' > e$, subject to the constraint that the width $w' = e' - s' + 1 \leq L$. In this case, the minimum length when using the superset token t'_k would be:

$$K^s_{t'_k} = pl[s'-1] + bpl[e'+1] + 1, \tag{3}$$

which is the path length to get to the byte before t'_k, plus the path length from the end of the data backwards to the byte after t'_k, plus 1 for the token t'_k itself.

We retain a list of the widths of the tokens ending at each byte.[7] The set of superset tokens S can be found by examining the potential e', and then seeing if the tokens ending at e' form a strict superset. Similar to the previous case, we can compute the minimum increase from replacing t_k with a superset token by taking the minimum increase over the superset tokens S:

$$MI^s_{kd} = \min_{t'_k \in S} K^s_{t'_k} - K_d. \tag{4}$$

We then aggregate over the documents to get the overall increase for each $t \in \mathcal{V}$:

$$MI_t = \sum_{d \in \mathcal{C}} \sum_{k=1|t_k=t}^{K_d} \min(MI^b_{kd}, MI^s_{kd}). \tag{5}$$

One iteration of this vocabulary construction procedure will have complexity $O(nL^2)$.[7]

3.3 Connecting PathPiece and Unigram

We note a connection between PathPiece and Unigram. In Unigram, the probability of a segmentation $t_1, \ldots, t_{K_d}$ is the product of the unigram token probabilities $p(t_k)$:

$$P(t_1, \ldots, t_{K_d}) = \prod_{k=1}^{K_d} p(t_k). \tag{6}$$

Taking the negative log of this product converts the objective from maximizing the likelihood to minimizing the sum of $-\log(p(t_k))$ terms. While Unigram is solved by the Viterbi (1967) algorithm, it can also be solved by a weighted version of PathPiece with weights of $-\log(p(t_k))$. Conversely, a solution minimizing the number of tokens can be found in Unigram by taking all $p(t_k) := 1/|\mathcal{V}|$.

4 Experiments

We used the Pile corpus (Gao et al., 2020; Biderman et al., 2022) for language model pre-training, which contains 825GB of English text data from 22 high-quality datasets. We constructed the tokenizer vocabularies over the MiniPile dataset (Kaddour, 2023), a 6GB subset of the Pile. We use the MosaicML Pretrained Transformers (MPT) decoder-only language model architecture.[8] Appendix B gives the full set of model parameters, and Appendix D discusses model convergence.

[7] See the expanded explanation in Appendix A for details.

[8] https://github.com/mosaicml/llm-foundry

4.1 Downstream Evaluation Tasks

To evaluate and analyze the performance of our tokenization process, we select 10 benchmarks from `lm-evaluation-harness` (Gao et al., 2023).[9] These are all multiple-choice tasks with 2, 4, or 5 options, and were run with 5-shot prompting. We use arc_easy (Clark et al., 2018), copa (Brassard et al., 2022), hendrycksTests-marketing (Hendrycks et al., 2021), hendrycksTests-sociology (Hendrycks et al., 2021), mathqa (Amini et al., 2019), piqa (Bisk et al., 2019), qa4mre_2013 (Peñas et al., 2013), race (Lai et al., 2017), sciq (Welbl et al., 2017), and wsc273 (Levesque et al., 2012). Appendix C gives a full description of these tasks.

4.2 Tokenization Stage Variants

We conduct the 18 experimental variants listed in Table 1, each repeated at the vocabulary sizes of 32,768, 40,960, and 49,152.[10] For baseline vocabulary creation methods, we used BPE, Unigram, WordPiece, and SaGe. We also consider two variants of PATHPIECE where ties in the shortest path are broken either by the longest token (PATHPIECEL), or randomly (PATHPIECER). For the vocabulary initialization required by PATHPIECE and SaGe, we experimented with the most common n-grams, as well as with a large initial vocabulary trained with BPE or Unigram. We also varied the pre-tokenization schemes for PATHPIECE and SaGe, using either no pre-tokenization or combinations of "FirstSpace", "Space", and "Digit" described in §2.1. Tokenizers usually use the same segmentation approach used in vocabulary construction. PATHPIECEL's shortest path segmentation can be used with any vocabulary, so we apply it to vocabularies trained by BPE and Unigram. We also apply a Greedy left-to-right longest-token segmentation approach to these vocabularies.

[9] https://github.com/EleutherAI/lm-evaluation-harness

[10] These sizes were selected because vocabularies in the 30k to 50k range are the most common amongst language models within the HuggingFace Transformers library, https://huggingface.co/docs/transformers/. Ali et al. (2024) recently examined the effect of vocabulary sizes and found 33k and 50k sizes performed better on English language tasks than larger sizes.

5 Results

Table 1 reports the downstream performance across all our experimental settings.[11] A random baseline for these 10 tasks yields 32%. The OVERALL AVG column indicates the average results over the three vocabulary sizes. The RANK column refers to the rank of each variant with respect to the OVERALL AVG column (Rank 1 is best), which we will sometimes use as a succinct way to refer to a variant.

5.1 Vocabulary Size

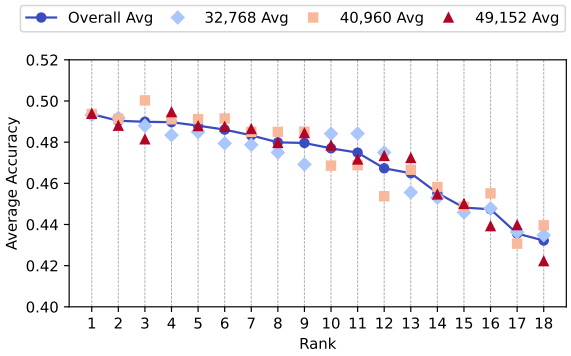


Figure 1: Effect of vocabulary size on downstream performance. For each tokenizer variant, we show the overall average, along with the three averages by vocabulary size, labeled according to the ranks in Table 1.

Figure 1 gives the overall average, along with the individual averages, for each of the three vocabulary sizes for each variant, labeled according to the rank from Table 1. We observe that there is a high correlation between downstream performance at different vocabulary sizes. The pairwise R^2 values for the accuracy of the 32,768 and 40,960 runs was 0.750; between 40,960 and 49,152 it was 0.801; and between 32,768 and 49,152 it was 0.834. This corroborates the effect shown graphically in Figure 1 that vocabulary size is not a crucial decision over this range of sizes. Given this high degree of correlation, we focus our analysis on the overall average accuracy. This averaging removes some of the variance amongst individual language model runs. Thus, unless specified otherwise, our analyses present performance averaged over vocabulary sizes.

[11] The same table sorted by rank is in Table 10 of Appendix G. The comprehensive results for the ten downstream tasks, for each of the 350M parameter models, are given in Appendix G.

Rank	Vocab Constr	Init Voc	Pre-tok	Segment	Overall	32,768	40,960	49,152
1	PathPieceL	BPE	FirstSpace	PathPieceL	**49.4**	**49.3**	49.4	49.4
9		Unigram	FirstSpace		48.0	47.0	48.5	48.4
15		n-gram	FirstSpDigit		44.8	44.6	44.9	45.0
16		n-gram	FirstSpace		44.7	44.8	45.5	43.9
2	Unigram		FirstSpace	Likelihood	49.0	49.2	49.1	48.8
7				Greedy	48.3	47.9	48.5	48.6
17				PathPieceL	43.6	43.6	43.1	44.0
3	BPE		FirstSpace	Merge	49.0	49.0	**50.0**	48.1
4				Greedy	49.0	48.3	49.1	**49.5**
13				PathPieceL	46.5	45.6	46.7	47.2
5	WordPiece		FirstSpace	Greedy	48.8	48.5	49.1	48.8
6	SaGe	BPE	FirstSpace	Greedy	48.6	48.0	49.2	48.8
8		n-gram	FirstSpace		48.0	47.5	48.5	48.0
10		Unigram	FirstSpace		47.7	48.4	46.9	47.8
11		n-gram	FirstSpDigit		47.5	48.4	46.9	47.2
12	PathPieceR	n-gram	SpaceDigit	PathPieceR	46.7	47.5	45.4	47.3
14			FirstSpDigit		45.5	45.3	45.8	45.5
18			None		43.2	43.5	44.0	42.2
	Random				32.0	32.0	32.0	32.0

Table 1: Summary of 350M parameter model downstream accuracy (%) across 10 tasks. The "Overall" column averages across the three vocabulary sizes. The "Rank" column refers to the Overall column, best to worst.

5.2 Overall performance

To determine which of the differences in the overall average accuracy in Table 1 are statistically significant, we conduct a one-sided Wilcoxon signed-rank test (Wilcoxon, 1945) on the paired differences of the 30 accuracy scores (three vocabulary sizes over ten tasks), for each pair of variants. All p-values reported in this paper use this test.

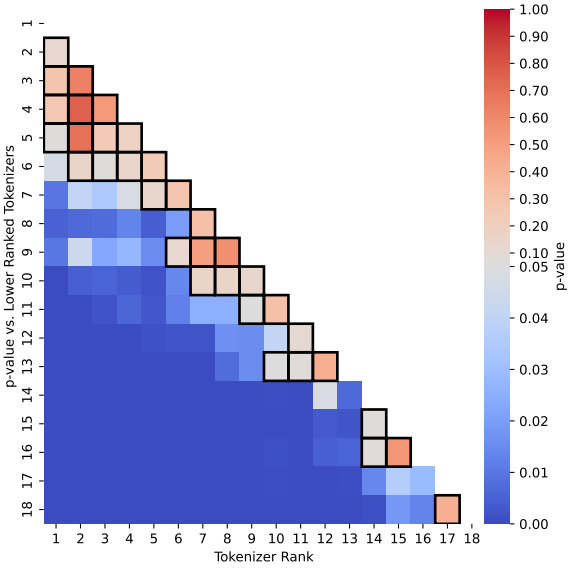


Figure 2: Pairwise p-values for 350M model results. Boxes outlined in black represent $p > 0.05$. The top 6 tokenizers are all competitive, and there is no statistically significantly best approach.

Figure 2 displays all pairwise p-values in a color map. Each column designates a tokenization variant by its rank in Table 1, compared to all the ranks below it. A box is outlined in black if $p > 0.05$, where we cannot reject the null. While PATHPIECEL-BPE had the highest overall average on these tasks, the top five tokenizers, PATHPIECEL-BPE, Unigram, BPE, BPE-Greedy, and WordPiece do not have any other row in Figure 2 significantly different from them. Additionally, SaGe-BPE (rank 6) is only barely worse than PATHPIECEL-BPE ($p = 0.047$), and should probably be included in the list of competitive tokenizers. Thus, our first key result is that there is no tokenizer algorithm better than all others to a statistically significant degree.

All the results reported thus far are for language models with identical architectures and 350M parameters. To examine the dependency on model size, we trained larger models of 1.3B parameters for six of our experiments, and 2.4B parameters for four of them. In the interest of computational time, these larger models were only trained with a single vocabulary size of 40,960. In Figure 6 in subsection 6.4, we report models' average performance across 10 tasks. See Figure 7 in Appendix D for an example checkpoint graph at each model size. The main result from these models is that the relative performance of the tokenizers does vary by model size, and that there is a group of high performing to-

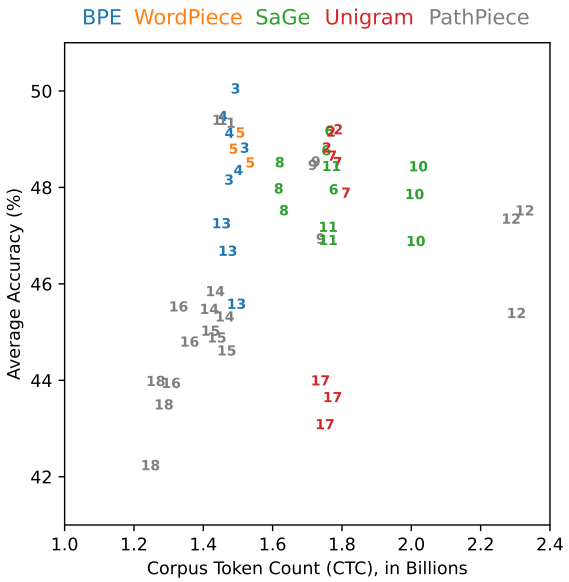


Figure 3: Effect of corpus token count (CTC) vs average accuracy of individual vocabulary sizes.

kenizers that yield comparable results. This aligns with our finding that the top six tokenizers are not statistically better than one another at the 350M model size.

5.3 Corpus Token Count vs Accuracy

Figure 3 shows the corpus token count (CTC) versus the accuracy of each vocabulary size, given in Table 11. We do not find a straightforward relationship between the two. Ali et al. (2024) recently examined the relationship between CTC and downstream performance for three different tokenizers, and also found it was not correlated on English language tasks.

The two models with the highest CTC are PATHPIECE with Space pre-tokenization (12), which is to be expected given each space is its own token, and SaGe with an initial Unigram vocabulary (10). The Huggingface Unigram models in Figure 3 had significantly higher CTC than the corresponding BPE models, unlike Bostrom and Durrett (2020) and Gow-Smith et al. (2022), which report a difference of only a few percent with SentencePiece Unigram. Ali et al. (2024) point out that due to differences in pre-processing, the Huggingface Unigram tokenizer behaves quite differently than the SentencePiece Unigram tokenizer, which may explain this discrepancy.

In terms of accuracy, PATHPIECE with no pre-tokenization (18) and Unigram with PATHPIECE segmentation (17) both did quite poorly. Notably,

Comparison	Pearson Correlation
CTC and Ave Acc	0.241
Rényi Eff and Ave Acc (α=1.5)	−0.221
Rényi Eff and Ave Acc (α=2.0)	−0.169
Rényi Eff and Ave Acc (α=2.5)	−0.151
Rényi Eff and Ave Acc (α=3.0)	−0.144
Rényi Eff and Ave Acc (α=3.5)	−0.141
CTC and Rényi Eff (α=2.5)	−0.891

Table 2: Pearson Correlation of CTC and Average Accuracy, or Rényi efficiency for various orders α with Average Accuracy, or CTC and Rényi efficiency at $\alpha = 2.5$.

the range of CTC is quite narrow within each vocabulary construction method, even while changes in pre-tokenization and segmentation lead to significant accuracy differences. While there are confounding factors present in this chart (e.g., pre-tokenization, vocabulary initialization, and that more tokens allow for additional computations by the downstream model) it is difficult to discern any trend that lower CTC leads to improved performance. If anything, there seems to be an inverted U-shaped curve with respect to the CTC and downstream performance. The Pearson correlation coefficient between CTC and average accuracy was found to be 0.241. Given that a lower CTC value signifies greater compression, this result suggests a weak negative relationship between the amount of compression and average accuracy.

Zouhar et al. (2023a) introduced an information-theoretic measure based on Rényi efficiency that correlates with downstream performance for their application.[12] It has an order parameter α, with a recommended value of 2.5. We present the Rényi efficiencies and CTC for all models in Table 11 in Appendix G, and summarize their Pearson correlation with average accuracy in Table 2. For the data of Figure 3, all the correlations for various α also have a weak negative association. They are slightly less negative than the association for CTC, although it is not nearly as large as the benefit they saw over sequence length in their application. We note the strong relationship between compression and Rényi efficiency, as the Pearson correlation of CTC and Rényi efficiency with α=2.5 is −0.891.

By varying aspects of BPE, Gallé (2019) and Goldman et al. (2024) suggests we should expect downstream performance to decrease with CTC, while in contrast Ali et al. (2024) did not find a

[12] Except, so far, for a family of adversarially-created tokenizers (Cognetta et al., 2024).

strong relation when varying the tokenizer. Our extensive results varying a number of stages of tokenization suggest it is not *inherently* beneficial to use fewer tokens. Rather, the particular way that the CTC is varied can lead to different conclusions.

6 Analysis

We now analyze the results across the various experiments in a more controlled manner. Our experiments allow us to examine changes in each stage of tokenization, holding the rest constant, revealing design decisions making a significant difference.[13]

6.1 Pre-tokenization

For PATHPIECER with an n-gram initial vocabulary, we can isolate pre-tokenization. PATHPIECE is efficient enough to process entire documents with no pre-tokenization, giving it full freedom to minimize the corpus token count (CTC).

Adding pre-tokenization constrains PATHPIECE's ability to minimize tokens, giving a natural way to vary the number of tokens. Figure 4 shows that PATHPIECE minimizes the number of tokens used over a corpus when trained with no pre-tokenization (18). The other variants restrict spaces to either be the first character of a token (14), or their own token (12).[14] Consider the example PATHPIECE tokenization in Table 3 for the three pre-tokenization methods. The NONE mode uses the word-boundary-spanning tokens "`ation␣is`", "`␣to␣b`", and "`e␣$`". The lack of morphological alignment demonstrated in this example is likely more important to downstream model performance than a simple token count.

In Figure 4 we observe a statistically significant increase in overall accuracy for our downstream tasks, as a function of CTC. Gow-Smith et al. (2022) found that Space pre-tokenization lead to worse performance, while removing the spaces entirely helps[15]. Thus, this particular result may be specific to PATHPIECER.

6.2 Vocabulary Construction

One way to examine the effects of vocabulary construction is to compare the resulting vocabularies of top-down methods trained using an initial vocabulary to the method itself. Figure 5 presents an

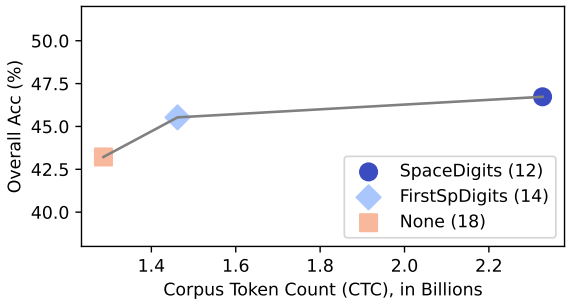


Figure 4: The impact of pre-tokenization on Corpus Token Count (CTC) and Overall Accuracy. Ranks in parentheses refer to performance in Table 1.

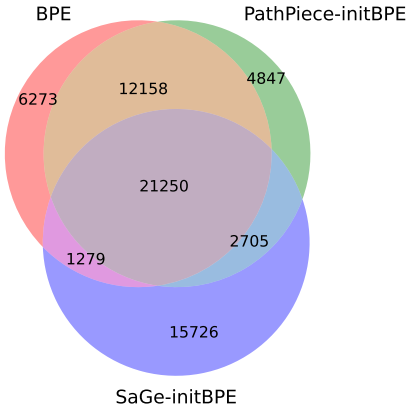


Figure 5: Venn diagram comparing 40,960 token vocabularies of BPE, PathPieceL and SaGe – the latter two were both initialized from a BPE vocabulary of 262,144.

area-proportional Venn diagram of the overlap in 40,960-sized vocabularies between BPE (6) and variants of PATHPIECEL (1) and SaGe (6) that were trained using an initial BPE vocabulary of size $2^{18} = 262,144$.[16] While BPE and PATHPIECEL overlap considerably, SaGe produces a more distinct set of tokens.

6.3 Initial Vocabulary

PATHPIECE, SaGe, and Unigram all require an initial vocabulary.[17] For PATHPIECE and SaGe, we experimented with initial vocabularies of size 262,144 constructed from either the most frequent n-grams, or trained using either BPE or Unigram. For PATHPIECEL, using a BPE initial vocabulary (1) is statistically better than both Unigram (9) and n-grams (16), with $p \leq 0.01$. Using an n-gram

[13] Appendix E contains additional analysis

[14] These two runs also used Digit pre-tokenization where each digit is its own token.

[15] Although omitting the spaces entirely does not lead to a reversible tokenization as we have been considering.

[16] See Figure 12 in Appendix E.3 for analogous results for Unigram, which behaves similarly.

[17] The HuggingFace Unigram implementation starts with the one million top n-grams, but sorted according to the count times the length of the token, introducing a bias toward longer tokens.

Rank	Pre-tokenization	Example
12	SpaceDigit	`The ␣ valuation ␣ is ␣ estimated ␣ to ␣ be ␣ $ 2 1 3 M`
14	FirstSpDigit	`The ␣valuation ␣is ␣estimated ␣to ␣be ␣$ 2 1 3 M`
18	None	`The ␣valu ation␣is ␣estimated ␣to␣b e␣$ 2 1 3 M`

Table 3: Example PATHPIECE tokenizations of "The valuation is estimated to be $213M"; vocabulary size of 32,768.

initial vocabulary leads to the lowest performance, with statistical significance. Comparing ranks 6, 8, and 10 reveals the same pattern for SaGe, although the difference between 8 and 10 is not significant.

6.4 Effect of Model Size

To examine the dependency on model size, we build larger models of 1.3B parameters for 6 of our experiments, and 2.4B parameters for 4 of them. These models were trained over the same 200 billion tokens. In the interest of computational time, these larger models were only run at a single vocabulary size of 40,960. The average results over the 10 task accuracies for these models is given in Figure 6. See Table 14 in Appendix G for the numerical values.

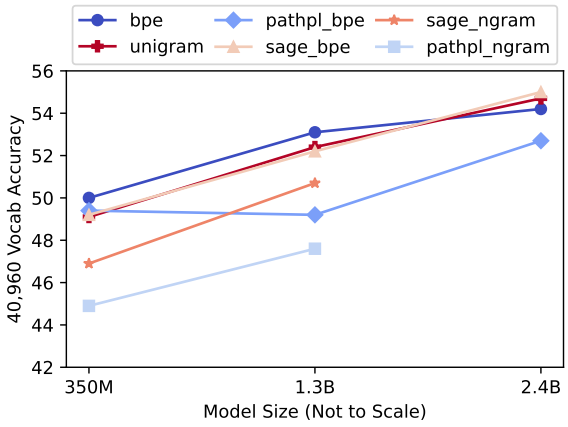


Figure 6: 40,960 vocab average accuracy at various models sizes

It is noteworthy from the prevalence of crossing lines in Figure 6 that the relative performance of the tokenizers do vary by model size, and that there is a group of tokenizers that are trading places being at the top for various model sizes. This aligns with our observation that the top 6 tokenizers were within the noise, and not significantly better than each other in the 350M models.

7 Conclusion

We investigate the hypothesis that reducing the corpus token count (CTC) would improve downstream performance, as suggested by Gallé (2019) and Goldman et al. (2024) when they varied aspects of BPE. When comparing CTC and downstream accuracy across all our experimental settings in Figure 3, we do not find a clear relationship between the two. We expand on the findings of Ali et al. (2024) who did not find a strong relation when comparing 3 tokenizers, as we run 18 experiments varying the tokenizer, initial vocabulary, pre-tokenizer, and inference method. Our results suggest compression is not a straightforward explanation of what makes a tokenizer effective.

Finally, this work makes several practical contributions: (1) vocabulary size has little impact on downstream performance over the range of sizes we examined (§5.1); (2) five different tokenizers all perform comparably, with none outperforming at statistical significance (§5.2); (3) BPE initial vocabularies work best for top-down vocabulary construction (§6.3). To further encourage research in this direction, we make all of our trained vocabularies publicly available, along with the model weights from our 64 language models.

Limitations

The objective of this work is to offer a comprehensive analysis of the tokenization process. However, our findings were constrained to particular tasks and models. Given the degrees of freedom, such as choice of downstream tasks, model, vocabulary size, etc., there is a potential risk of inadvertently considering our results as universally applicable to all NLP tasks; results may not generalize to other domains of tasks.

Additionally, our experiments were exclusively with English language text, and it is not clear how these results will extend to other languages. In particular, our finding that pre-tokenization is crucial for effective downstream accuracy is not applicable to languages without space-delimited words.

We conducted experiments for three district vocabulary sizes, and we reported averaged results across these experiments. With additional compute resources and time, it could be beneficial to con-

duct further experiments to gain a better estimate of any potential noise. For example, in Figure 7 of Appendix D, the 100k checkpoint at the 1.3B model size is worse than expected, indicating that noise could be an issue.

Finally, the selection of downstream tasks can have a strong impact on results. To allow for meaningful results, we attempted to select tasks that were neither too difficult nor too easy for the 350M parameter models, but other choices could lead to different outcomes. There does not seem to be a good, objective criteria for selecting a finite set of task to well-represent global performance.

Ethics Statement

We have used the commonly used public dataset The Pile, which has not undergone a formal ethics review (Biderman et al., 2022). Our models may include biases from the training data.

Our experimentation has used considerable energy. Each 350M parameter run took approximately 48 hours on (4) p4de nodes, each containing 8 NVIDIA A100 GPUs. We trained 62 models, including the 8 RandTrain runs in Appendix F. The (6) 1.3B parameters models took approximately 69 hours to train on (4) p4de nodes, while the (4) 2.4B models took approximately 117 hours to train on (8) p4de nodes. In total, training required 17,304 hours of p4de usage (138,432 GPU hours).

Acknowledgments

Thanks to Charles Lovering at Kensho for his insightful suggestions, and to Michael Krumdick, Mike Arov, and Brian Chen at Kensho for their help with the language model development process. This research was supported in part by the Israel Science Foundation (grant No. 1166/23). Thanks to an anonymous reviewer who pointed out the large change in CTC when comparing Huggingface BPE and Unigram, in contrast to the previous literature using the SentencePiece implementations (Kudo and Richardson, 2018).

References

Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Schulze Buschhoff, Charvi Jain, Alexander Arno Weber, Lena Jurkschat, Hammam Abdelwahab, Chelsea John, Pedro Ortiz Suarez, Malte Ostendorff, Samuel Weinbach, Rafet Sifa, Stefan Kesselheim, and Nicolas Flores-Herr. 2024. Tokenizer choice for llm training: Negligible or crucial?

Aida Amini, Saadia Gabriel, Peter Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. 2019. Mathqa: Towards interpretable math word problem solving with operation-based formalisms.

Thomas Bauwens and Pieter Delobelle. 2024. BPE-knockout: Pruning pre-existing BPE tokenisers with backwards-compatible morphological semi-supervision. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5810–5832, Mexico City, Mexico. Association for Computational Linguistics.

Stella Biderman, Kieran Bicheno, and Leo Gao. 2022. Datasheet for the pile. *CoRR*, abs/2201.07311.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2019. Piqa: Reasoning about physical commonsense in natural language.

Kaj Bostrom and Greg Durrett. 2020. Byte pair encoding is suboptimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4617–4624, Online. Association for Computational Linguistics.

Gerlof Bouma. 2009. Normalized (pointwise) mutual information in collocation extraction. *Proceedings of GSCL*, 30:31–40.

Ana Brassard, Benjamin Heinzerling, Pride Kavumba, and Kentaro Inui. 2022. Copa-sse: Semi-structured explanations for commonsense reasoning.

Pavel Chizhov, Catherine Arnett, Elizaveta Korotkova, and Ivan P. Yamshchikov. 2024. Bpe gets picky: Efficient vocabulary refinement during tokenizer training.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457.

Marco Cognetta, Vilém Zouhar, Sangwhan Moon, and Naoaki Okazaki. 2024. Two counterexamples to tokenization and the noiseless channel. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 16897–16906, Torino, Italia. ELRA and ICCL.

Pavlos S. Efraimidis. 2010. Weighted random sampling over data streams. *CoRR*, abs/1012.0256.

Philip Gage. 1994. A new algorithm for data compression. *C Users J.*, 12(2):23–38.

Matthias Gallé. 2019. Investigating the effectiveness of BPE: The power of shorter sequences. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 1375–1381, Hong Kong, China. Association for Computational Linguistics.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The pile: An 800gb dataset of diverse text for language modeling.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A framework for few-shot language model evaluation.

Omer Goldman, Avi Caciularu, Matan Eyal, Kris Cao, Idan Szpektor, and Reut Tsarfaty. 2024. Unpacking tokenization: Evaluating text compression and its correlation with model performance.

Edward Gow-Smith, Dylan Phelps, Harish Tayyar Madabushi, Carolina Scarton, and Aline Villavicencio. 2024. Word boundary information isn't useful for encoder language models. In *Proceedings of the 9th Workshop on Representation Learning for NLP (RepL4NLP-2024)*, pages 118–135, Bangkok, Thailand. Association for Computational Linguistics.

Edward Gow-Smith, Harish Tayyar Madabushi, Carolina Scarton, and Aline Villavicencio. 2022. Improving tokenisation by alternative treatment of spaces. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11430–11443, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Gregory Grefenstette. 1999. *Tokenization*, pages 117–133. Springer Netherlands, Dordrecht.

Ximena Gutierrez-Vasques, Christian Bentz, Olga Sozinova, and Tanja Samardzic. 2021. From characters to words: the turning point of BPE merges. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 3454–3468, Online. Association for Computational Linguistics.

Xuanli He, Gholamreza Haffari, and Mohammad Norouzi. 2020. Dynamic programming encoding for subword segmentation in neural machine translation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 3042–3051, Online. Association for Computational Linguistics.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding.

Valentin Hofmann, Janet Pierrehumbert, and Hinrich Schütze. 2021. Superbizarre is not superb: Derivational morphology improves BERT's interpretation of complex words. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3594–3608, Online. Association for Computational Linguistics.

Valentin Hofmann, Hinrich Schuetze, and Janet Pierrehumbert. 2022. An embarrassingly simple method to mitigate undesirable properties of pretrained language model tokenizers. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 385–393, Dublin, Ireland. Association for Computational Linguistics.

Cassandra L Jacobs and Yuval Pinter. 2022. Lost in space marking. *arXiv preprint arXiv:2208.01561*.

Jean Kaddour. 2023. The minipile challenge for data-efficient language models.

Stav Klein and Reut Tsarfaty. 2020. Getting the ##life out of living: How adequate are word-pieces for modelling complex morphology? In *Proceedings of the 17th SIGMORPHON Workshop on Computational Research in Phonetics, Phonology, and Morphology*, pages 204–209, Online. Association for Computational Linguistics.

Taku Kudo. 2018. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 66–75, Melbourne, Australia. Association for Computational Linguistics.

Taku Kudo and John Richardson. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium. Association for Computational Linguistics.

Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. 2017. Race: Large-scale reading comprehension dataset from examinations.

Hector J. Levesque, Ernest Davis, and Leora Morgenstern. 2012. The winograd schema challenge. In *13th International Conference on the Principles of Knowledge Representation and Reasoning, KR 2012*, Proceedings of the International Conference on Knowledge Representation and Reasoning, pages 552–561. Institute of Electrical and Electronics Engineers Inc. 13th International Conference on the Principles of

Knowledge Representation and Reasoning, KR 2012 ; Conference date: 10-06-2012 Through 14-06-2012.

Haoran Lian, Yizhe Xiong, Jianwei Niu, Shasha Mo, Zhenpeng Su, Zijia Lin, Peng Liu, Hui Chen, and Guiguang Ding. 2024. Scaffold-bpe: Enhancing byte pair encoding with simple and effective scaffold token removal.

Davis Liang, Hila Gonen, Yuning Mao, Rui Hou, Naman Goyal, Marjan Ghazvininejad, Luke Zettlemoyer, and Madian Khabsa. 2023. XLM-V: Overcoming the vocabulary bottleneck in multilingual masked language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13142–13152, Singapore. Association for Computational Linguistics.

Tomasz Limisiewicz, Jiří Balhar, and David Mareček. 2023. Tokenization impacts multilingual language modeling: Assessing vocabulary allocation and overlap across languages. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 5661–5681, Toronto, Canada. Association for Computational Linguistics.

Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. 2021. Between words and characters: A brief history of open-vocabulary modeling and tokenization in nlp.

Tomas Mikolov, Ilya Sutskever, Anoop Deoras, Hai Son Le, Stefan Kombrink, and Jan Honza Černocký. 2011. Subword language modeling with neural networks. Preprint available at: https://api.semanticscholar.org/CorpusID:46542477.

Anselmo Peñas, Eduard Hovy, Pamela Forner, Álvaro Rodrigo, Richard Sutcliffe, and Roser Morante. 2013. Qa4mre 2011-2013: Overview of question answering for machine reading evaluation. In *CLEF 2013, LNCS 8138*, pages 303–320.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. 2020. BPE-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1882–1892, Online. Association for Computational Linguistics.

Jonne Saleva and Constantine Lignos. 2023. What changes when you randomly choose BPE merge operations? not much. In *Proceedings of the Fourth Workshop on Insights from Negative Results in NLP*, pages 59–66, Dubrovnik, Croatia. Association for Computational Linguistics.

Mike Schuster and Kaisuke Nakajima. 2012. Japanese and korean voice search. In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5149–5152.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.

Jasdeep Singh, Bryan McCann, Richard Socher, and Caiming Xiong. 2019. BERT is not an interlingua and the bias of tokenization. In *Proceedings of the 2nd Workshop on Deep Learning Approaches for Low-Resource NLP (DeepLo 2019)*, pages 47–55, Hong Kong, China. Association for Computational Linguistics.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models.

Omri Uzan, Craig W. Schmidt, Chris Tanner, and Yuval Pinter. 2024. Greed is all you need: An evaluation of tokenizer inference methods. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 813–822, Bangkok, Thailand. Association for Computational Linguistics.

A. Viterbi. 1967. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269.

Jeffrey S. Vitter. 1985. Random sampling with a reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57.

Johannes Welbl, Nelson F. Liu, and Matt Gardner. 2017. Crowdsourcing multiple choice science questions. *ArXiv*, abs/1707.06209.

F Wilcoxon. 1945. Individual comparisons by ranking methods. biom. bull., 1, 80–83.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. 2016. Google's neural machine translation system: Bridging the gap between human and machine translation.

Shaked Yehezkel and Yuval Pinter. 2023. Incorporating context into subword vocabularies. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 623–635, Dubrovnik, Croatia. Association for Computational Linguistics.

Vilém Zouhar, Clara Meister, Juan Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. 2023a. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5184–5207, Toronto, Canada. Association for Computational Linguistics.

Vilém Zouhar, Clara Meister, Juan Gastaldi, Li Du, Tim Vieira, Mrinmaya Sachan, and Ryan Cotterell. 2023b. A formal perspective on byte-pair encoding. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 598–614, Toronto, Canada. Association for Computational Linguistics.

A Expanded description of PATHPIECE

This section provides a self-contained explanation of PATHPIECE, expanding on the one in §3, with additional details on the vocabulary construction and complexity.

In order to design an optimal vocabulary $\mathcal{V}$, it is first necessary to know how the vocabulary will be used to tokenize. There can be no best vocabulary in the abstract. Thus, we first present a new lossless subword tokenizer PATHPIECE. This tokenization over our training corpus will provide the context to design a coherent vocabulary.

A.1 Tokenization for a given vocabulary

We work at the byte level, and require that all 256 single byte tokens are included in any given vocabulary $\mathcal{V}$. This avoids any out-of-vocabulary tokens by falling back to single bytes in the worst case.

Tokenization can be viewed as a compression problem, where we would like to tokenize text in a few tokens as possible. This has direct benefits, as it allows more text to fit in a given context window. A Minimum Description Length (MDL) argument can also be made that the tokenization using the fewest tokens best describes the data, although we saw in Subsection 6.1 this may not always hold in practice.

Tokenizers such as BPE and WordPiece make greedy decisions, such as choosing which pair of current tokens to merge to create a new one, which results in tokenizations that may use more tokens than necessary. In contrast, PATHPIECE will find an optimal tokenization by finding a shortest path through a Directed Acyclic Graph (DAG). Informally, each byte i of training data forms a node in the graph, and there is an edge if the w byte sequence ending at i is a token in $\mathcal{V}$.

An implementation of PATHPIECE is given in Algorithm 2, where input d is a text document of n bytes, $\mathcal{V}$ is a given vocabulary, and L is a limit on the maximum width of a token in bytes. It has complexity $O(nL)$, following directly from the two nested `for`-loops. It iterates over the bytes i in d, computing 4 values for each. It computes the shortest path length $pl[i]$ in tokens up to and including byte i, the width $wid[i]$ of a token with that shortest path length, and the solution count $sc[i]$ of optimal solutions found thus far with that shortest length. We also remember the valid tokens of width 2 or more ending at each location i in $vt[i]$, which will be used in the next section.

There will be multiple tokenizations with the same optimal length, so some sort of tiebreaker is needed. The longest token or a randomly selected token are obvious choices. We have presented the random tiebreaker method here, where a random solution is selected in a single pass in lines 29-32 of the listing using an idea from reservoir sampling (Vitter, 1985).

A backward pass through d constructs the optimal tokenization from the $wid[e]$ values from the forward pass.

A.2 Optimal Vocabulary Construction

A.2.1 Vocabulary Initialization

We will build an optimal vocabulary by starting from a large initial one, and sequentially omitting batches of tokens. We start with the most frequently occurring byte n-grams in a training corpus, of width 1 to L, or a large vocabulary trained by BPE or Unigram. We then add any single byte tokens that were not already included, making room by dropping the tokens with the lowest counts. In our experiments we used an initial vocabulary size of $|\mathcal{V}| = 2^{18} = 262,144$.

A.2.2 Increase from omitting a token

Given a PATHPIECE tokenization $t_1, \ldots, t_{K_d}$, $\forall d \in \mathcal{C}$ for training corpus $\mathcal{C}$, we would like to know the increase in the overall length of a tokenization $K = \sum_d K_d$ from omitting a given token t from our vocabulary, $\mathcal{V} \setminus \{t\}$ and recomputing the tokenization. Tokens with a low increase are good candidates to remove from the vocabulary (Kudo, 2018). However, doing this from scratch for each t would be a very expensive $O(nL|\mathcal{V}|)$ operation.

We make a simplifying assumption that allows us to compute these increases more efficiently. We omit a specific token t_k in the tokenization of document d, and compute the minimum increase MI_{kd}

Algorithm 2 PATHPIECE segmentation.

```
procedure PATHPIECE(d, V, L)
    n ← len(d)                                  ▷ document length
    for i ← 1, n do
        wid[i] ← 0                              ▷ shortest path token
        pl[i] ← ∞                               ▷ shortest path len
        sc[i] ← 0                               ▷ solution count
        vt[i] ← [ ]                             ▷ valid token list
    for e ← 1, n do                             ▷ token end
        for w ← 1, L do                         ▷ token width
            s ← e − w + 1                       ▷ token start
            if s ≥ 1 then                       ▷ s in range
                t ← d[s : e]                    ▷ token
                if t ∈ V then
                    if s = 1 then               ▷ 1 tok path
                        wid[e] ← w
                        pl[e] ← 1
                        sc[e] ← 1
                    else
                        if w ≥ 2 then
                            vt[e].append(w)
                        nl ← pl[s − 1] + 1
                        if nl < pl[e] then
                            pl[e] ← nl
                            wid[e] ← w
                            sc[e] ← 1
                        else if nl = pl[e] then
                            sc[e] ← sc[e] + 1
                            r ← rand()
                            if r ≤ 1/sc[e] then
                                wid[e] ← w
    T ← [ ]                                     ▷ output token list
    e ← n                                       ▷ start at end of d
    while e ≥ 1 do
        w ← wid[e]                              ▷ width of short path tok
        s ← e − w + 1                           ▷ token start
        t ← d[s : e]                            ▷ token
        T.append(t)
        e ← e − w                               ▷ back up a token
    return reversed(T)                          ▷ reverse order
```

in K_d from not having that token t_k in the tokenization of d. We then aggregate over the documents to get the overall increase for t:

$$MI_t = \sum_{d \in \mathcal{C}} \sum_{k=1|t_k=t}^{K_d} MI_{kd}. \tag{7}$$

This is similar to computing the increase from $\mathcal{V} \setminus \{t\}$, but ignores interaction effects from having several occurrences of the same token t close to each other in a given document.

With PATHPIECE, it turns out we can compute the minimum increase in tokenization length without actually recomputing the tokenization. Any tokenization not containing t_k must either contain a token boundary somewhere inside of t_k breaking it in two, or it must contain a token that entirely contains t_k as a superset. Our approach will be to enumerate all the occurrences for these two cases, and to find the minimum increase MI_{kd} overall.

Before considering these two cases, there is a shortcut that often tells us that there would be no increase due to omitting t_k ending at index e. We computed the solution count vector $sc[e]$ when running Algorithm 2. If $sc[e] > 1$ for a token ending at e, then the backward pass could simply select one of the alternate optimal tokens, and find an overall tokenization of the same length.

Let t_k start at index s and end at index e, inclusive. Remember that path length $pl[i]$ represents the number of tokens required for shortest path up to and including byte i. We can also run Algorithm 2 backwards on d, computing a similar vector of backwards path lengths $bpl[i]$, representing the number of tokens on a path from the end of the data up to and including byte i. The overall minimum length of a tokenization with a token boundary after byte j is thus:

$$K^b_j = pl[j] + bpl[j+1]. \tag{8}$$

We have added an extra constraint on the shortest path, that there is a break at j, so clearly $K^{br}_j \geq pl[n]$. The minimum increase for the case of having a token boundary within t_k is thus:

$$MI^b_{kd} = \min_{j=s,\ldots,e-1} K^b_j - pl[n]. \tag{9}$$

Each token t_k will have no more than $L-1$ potential internal breaks, so the complexity of computing MI^b_{kd} is $O(L)$.

The minimum increase from omitting t_k could also be on a tokenization containing a strict superset of t_k. Let this superset token be t'_k, with start s' and end e' inclusive. To be a strict superset jumping over t_k, we must have $s' < s$ and $e' \geq e$, or $s' \leq s$ and $e' > e$, subject to the constraint that the width $w' = e' - s' + 1 \leq L$. In this case, the minimum length of using the superset token t'_k would be:

$$K^s_{t'_k} = pl[s'-1] + bpl[e'+1] + 1, \tag{10}$$

which is the path length to get to the byte before t'_k, plus the path length go backwards to the byte after t'_k, plus 1 for the token t'_k itself.

We remembered a list of the widths of the tokens ending at each byte, $vt[e]$ in Algorithm 2. The set of superset tokens S can be found by examining the $O(L)$ potential e', and then seeing if the $w' \in vt[e']$ give tokens forming a strict superset. There are $O(L)$ potential tokens ending at e' in $vt[e']$, so the overall complexity of finding the superset tokens is $O(L^2)$

Similar to the previous case, we can compute the minimum increase from replacing t_k with a superset token by taking the minimum increase over the superset tokens:

$$MI^s_{kd} = \min_{t'_k \in S} K^s_{t'_k} - pl[n]. \quad (11)$$

Finally, the overall minimum increase MI_{kd} from omitting t_k is simply

$$MI_{kd} = \min(MI^b_{kd}, MI^s_{kd}). \quad (12)$$

When aggregating over all t_k according to eq (7), one iteration of the vocabulary construction procedure will have complexity $O(nL^2)$.

B Language Model Parameters

The 350M parameter models were trained using the MPT architecture[18] with the following parameters:

```
# Model
model:
  name: mpt_causal_lm
  init_deice: meta
  d_model: 1024
  n_heads: 16
  n_layers: 24
  expansion_ratio: 4
  max_seq_len: 2048
  attn_config:
    alibi: true
    attn_impl: triton
    clip_qkv: 6

# Optimization
device_eval_batch_size: 5
device_train_microbatch_size: 32
global_train_batch_size: 1024 # ~2M tokens
max_duration: 100000ba # ~200B tokens

optimizer:
  name: decoupled_adamw
  lr: 3.0e-4
  betas:
  - 0.9
  - 0.95
  eps: 1.0e-08
  weight_decay: 0.0001

scheduler:
  name: cosine_with_warmup
  t_warmup: 0.05dur
  alpha_f: 0.1

# System
precision: amp_bf16

# Algos and Callbacks
algorithms:
  gradient_clipping:
    clipping_threshold: 1
    clipping_type: norm
```

The 1.3B parameter models simply changes:

```
d_model: 1024
```

The 2.4B parameter models updates:

```
d_model: 2560
n_heads: 20
n_layers: 32
```

C Description of Downstream Tasks

To evaluate the performance of our various tokenization experiments, we select ten competitive benchmarks from `lm-evaluation-harness` (Gao et al., 2023)[19], that we broadly categorize into three types of Question Answering (QA) tasks: Knowledge-based, Common-sense Reasoning and Context-based.

Knowledge Based Tasks Knowledge based tasks in this study expect LLMs to answer questions based on domain-specific internal retrieval. Our Knowledge-based baselines in this work include:

SciQ: The SciQ task, proposed by Welbl et al. (2017) contains a total of 13,679 science exam questions. The questions are in multiple-choice format with 4 answer options each. An additional text is provided as supporting evidence for a majority of the answers.

ARC (AI2 Reasoning Challenge): Clark et al. (2018) compiles grade-school level, multiple-choice science question dataset consists of 7,787 science exam questions that are split into "easy" and "hard" sets. For this study, we employ the easy set of 5,197 questions, each having 4 answer choices.

MathQA: Amini et al. (2019) introduce a dataset of math word problems that require LLMs to use their internal understanding of mathematical equations and arithmetic comprehension. Similar to SciQ, this dataset consists of 37k multiple-choice questions with the equations for each used annotated.

HendrycksTest: Hendrycks et al. (2021) provide a comprehensive suite of of multiple-choice tests for assessing text models in multi-task contexts. It comprises of 57 tasks such as elementary mathematics, US history, law of which we use the sociology and marketing tests.

Commonsense Reasoning Tasks These tasks assess the model's capability to infer and reason

[18] https://github.com/mosaicml/llm-foundry

[19] https://github.com/EleutherAI/lm-evaluation-harness

about everyday scenarios based on implicit knowledge.

COPA (Choice of Plausible Alternatives): COPA proposed by Brassard et al. (2022) is a benchmark for assessing progress in open-domain commonsense causal reasoning. It consists of 1000 questions where each question is composed of a premise and two alternatives. The task is to select the alternative that more plausibly has a causal relation with the premise.

PiQA (Physical Interaction Question Answering): Bisk et al. (2019) introduce a task that assess the understanding of physical commonsense by language models. Comprised of everyday situation with a preference for atypical solutions, this dataset is formulated as multiple choice question with two possible solutions choices for each question.

Winograd Schema Challenge: Levesque et al. (2012) define a task with a pair of sentences that differ only in one or two words and that contain a referential ambiguity that is resolved in opposite directions in the two sentences. This dataset of 273 tasks test language model understanding of the content of the text and disambiguation ability.

Context Based Tasks These tasks are reliant on understanding context and drawing conclusions from it.

RACE (Reading Comprehension from Examinations): RACE proposed by Lai et al. (2017) is a collection of English questions set aside to Chinese school students. Each item is divided into two parts, a passage that the student must read and a set of 4 potential answers, requiring extraction and reasoning capabilities.

QA4MRE (Question Answering for Machine Reading Evaluation): QA4MRE by Peñas et al. (2013) is a benchmark designed to resolve reading comprehension challenges. This task focuses on reading of single documents and identifying the answers to a set of questions. Questions are in the form of multiple choice with one correct option.

Our goal was to select tasks where a 350M parameter model could do significantly better than random chance, avoiding evaluation right at the noisier random threshold. We started with the tasks that had a non-zero random score (indicating multiple choice), and then chose tasks where BPE at a vocabulary size 40,960 could do well above random. In the end, the average accuracy across models was more than 15% above random on all tasks.

Note that in results tables we have shortened the name hendrycksTest-marketing to marketing, hendrycksTest-sociology to sociology, and qa4mre_2013 to qa4mre.

D Effect of model convergence

Each model was trained on around 200 billion tokens. Figure 7 gives a plot of the average accuracy for PathPieceL with a BPE initial vocabulary and a vocabulary size of 40,960 at various checkpoints in the language model training process. It also shows checkpoints for the larger 1.3B and 2.4B models discussed in the Limitations section. With the exception of the 100k checkpoint at 1.3B, the model appears to be continually improving. It is unclear why the 100k checkpoint did so poorly.

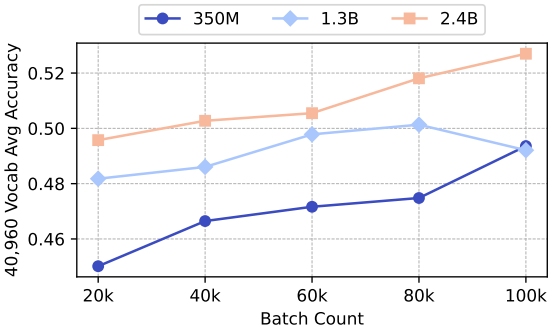


Figure 7: Checkpoint accuracy values for PathPieceL with an initial vocabulary from BPE and a vocabulary size of 40,960, evaluated at 5 checkpoints.

E Additional Analysis

Here we additional details for results from §6 that are just summarized in the text in the interest of space.

E.1 Segmentation

Tokenizers often use the segmentation strategy that is used in vocabulary construction. However, any vocabulary can also be used with PATHPIECE and with the greedy left-to-right segmentation methods.

We find that BPE works quite well with greedy segmentation (overall rank 4, insignificantly different from the top rank), but not with the shortest-path segmentation of PATHPIECEL (13).

Unigram, on the other hand, seems to be more tightly tied to its default maximum likelihood segmentation (2), which was significantly better than both Greedy (7) and PATHPIECEL (17).

E.2 Digit Pre-tokenization

We have two examples isolating Digit pre-tokenization, when a digit must always be its own token.

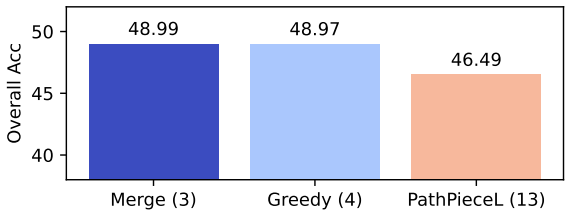


Figure 8: Segmentation of BPE.
Pairwise p-values between the pairs of runs are p(3,4)=0.52, p(3,13)=4.4e-5, p(4,13)=8.8e-6.

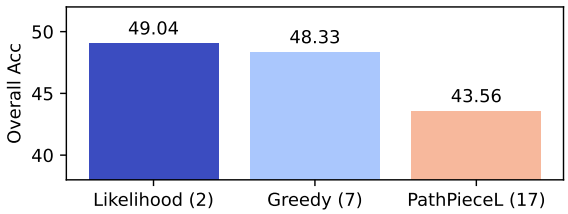


Figure 9: Segmentation of Unigram.
Pairwise p-values between the pairs of runs are p(2,7)=0.041, p(2,17)=2.9e-06, p(7,17)=2.9e-06

Figure 10 shows Digit hurts for Sage with an n-gram initial vocabulary, while Figure 11 shows no significant differences for PathPieceL, also with an n-gram initial vocabulary.

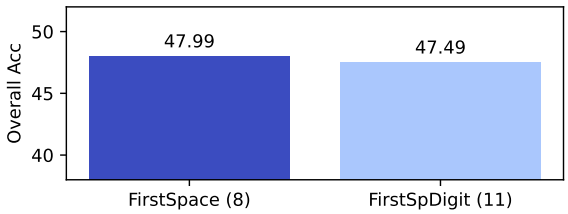


Figure 10: Pre-tokenization of Sage, n-gram initial, p=0.025.

With the exception of mathqa, none of our downstream tasks were particularly mathematical in nature. It is likely this makes it hard to make a definitive judgement on Digit with our experiments.

E.3 Vocabulary Construction

Figure 12 gives a Venn diagram of the overlap in vocabularies between Unigram, PathPieceL, and SaGe, when both PathPieceL and SaGe were constructed from a large initial vocabulary of size 262,144 from Unigram. As with Figure 5, we see that PathPiece is more similar to Unigram, while SaGe chose more distinct tokens.

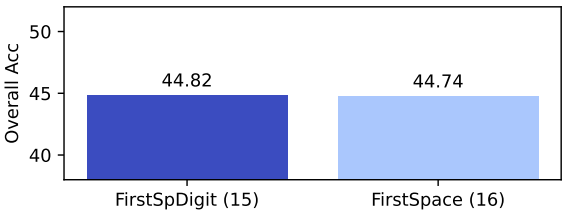


Figure 11: Pre-tokenization of PathPieceL n-gram, p=0.54.

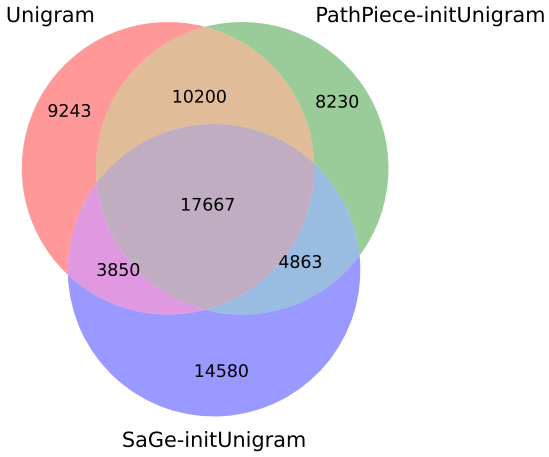


Figure 12: Venn diagrams comparing 40,960 token vocabularies of Unigram, PathPieceL and SaGe, where the latter two were both trained from a initial Unigram vocabulary of size 262,144

E.4 PathPiece tie breaking

The difference in tie breaking between choosing the longest token with PathPieceL versus choosing randomly with PathPieceR turns out not to be significant, as seen in in Figure 13.

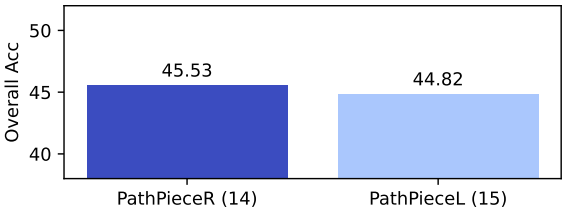


Figure 13: Tiebreaking PathPieceL vs PathPieceR with n-gram, p=0.067.

F RandTrain

None of our experiments completely isolate the effect of the vocabulary construction step. We created a new baseline random vocabulary construction approach, RandTrain, in an attempt to do so. It is meant to work with a top-down method like SaGe

or PathPieceL, and uses the same initial vocabulary, pre-tokenization, and segmentation as either of those, with a simple vocabulary construction algorithm.

We compute a count for each token in the vocabulary. For the top n-gram initial vocabulary it is simply the n-gram count from the training corpus. For a BPE initial vocabulary we tokenized the training corpus with BPE and the large initial vocabulary, and then use the occurrence counts of each token. We normalize these counts into target selection probabilities p_k for token t_k.

The RandTrain vocabulary construction process is simply to randomly sample our desired vocabulary size m of tokens from the initial vocabulary, proportionally to p_k, without replacement. Sampling without replacement is necessary to avoid have duplicate words in the vocabulary. Interestingly, this is not possible if there are any $p_k > 1/m$, which are termed infeasible or overweight items (Efraimidis, 2010). The intuition behind this is when selecting m items without replacement, it is not possible to select a given item more than once. So even if an item is always selected in a sample, the selection probability will be $p_k = 1/m$.

We sampled without replacement using the A-ES Algorithm described in Efraimidis (2010). A significant number the most common tokens in the vocabulary were infeasible and hence were unable to reach their target p_k. A token with a higher p_k is more likely to be sampled than a token with a lower one, but they may significantly differ from their target p_k.

We build 6 RandTrain models with 3 different types of pre-tokenization, and with Greedy segmentation to compare to SaGe, and PathPieceL segmentation to compare to PathPieceL. We only used a single vocabulary size of 40,960, so p-values are only computed on the 10 task accuracies, rather than the 30 used elsewhere. Task level accuracies are given in Table 6 and Table 7 in Appendix G.

Before comparing RandTrain to SaGe and PathPieceL, we will compare our RandTrain runs to each other, with different segmentation approaches. In Figure 14 and Figure 16 we have pairs of RandTrain runs that only vary by the segmentation method.

In line with Subsection E.1, Greedy performs significantly better than PathPieceL segmentation in all 3 cases. However, for the two cases with an n-gram initial vocabulary the PathPieceL segmentation did extremely poorly. The RandTrain

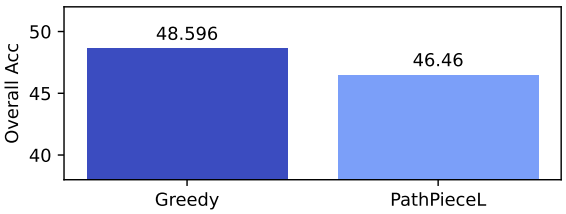


Figure 14: Comparison of Greedy and PathPieceL segmentation, with RandTrain vocabulary construction, BPE initial vocab, and FirstSpace pre-tokenization, p=0.0273

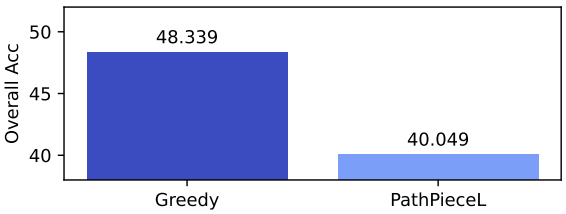


Figure 15: Comparison of Greedy and PathPieceL segmentation, with RandTrain vocabulary construction, n-gram initial vocab, and FirstSpace pre-tokenization, p=0.00195

vocabulary construction, n-gram initial vocabulary, and PathPieceL segmentation interact somehow to give accuracies well below any others.

This makes the comparison of RandTrain to PathPieceL less informative. We can see in Figure 17 that PathPieceL is significantly better than RandTrain with a BPE initial vocabulary.

However, the other two comparisons in Figure 18 are Figure 19 are not that meaningful. They are significantly better, but that is more about the weak baseline of RandTrain with PathPieceL segmentation than anything positive about PathPieceL.

The remaining comparison between SaGe and RandTrain is more interesting. In Figure 20 and Figure 21 SaGe was not significantly better than RandTrain, with a p-value of 0.0645.

The cases is even worse for the two n-gram initial vocabulary cases. In Figure 21 the p-value was a 0.688, and in Figure 22 RandTrain was actually better, although not significantly.

We saw in Table 1 that both PathPieceL-BPE and SaGe-BPE are effective tokenizers. In attempting to isolate the benefit from the vocabulary construction step, we see that PathPieceL-BPE outperforms our simple baseline. However, SaGe was unable to outperform the baseline, perhaps implying that RandTrain may actually be a simple but fairly effective vocabulary construction method.

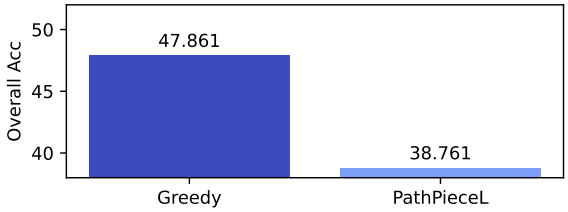


Figure 16: Comparison of Greedy and PathPieceL segmentation, with RandTrain vocabulary construction, n-gram initial vocab, and FirstSpaceDigit pre-tokenization, p=0.00293

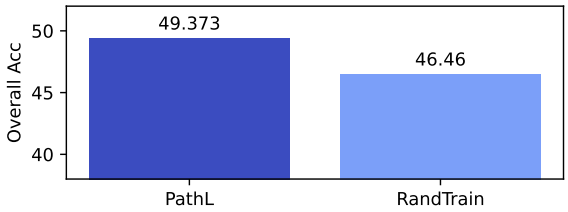


Figure 17: Comparison of PathPieceL and RandTrain, with BPE initial vocab, and FirstSpace pre-tokenization, p=0.0137

G Detailed Experimental Results

This section gives the detailed accuracy results for the 10 downstream evaluation tasks on each model that was trained. The tables are divided by the vocabulary size used, with Table 4 and Table 5 for 32,768; Table 6 and Table 7 for 40,960; and Table 8 and Table 9 for 49,152. The highest value or values (in the case of ties) are shown in bold. Table 10 show the same results as Table 1, but are sorted from best to worst by rank. The corpus token count (CTC), Rényi efficiencies, and average accuracies for the 54 runs in Figure 3 are given in Table 11.

The detailed accuracy results for our 1.3B parameter models, which were all performed at a single vocabulary size of 40,960, are given in Table 12 and Table 13. Average accuracy results for larger models of 1.3B and 2.4B parameters are given in Table 14. See §7 for more discussion of this table.

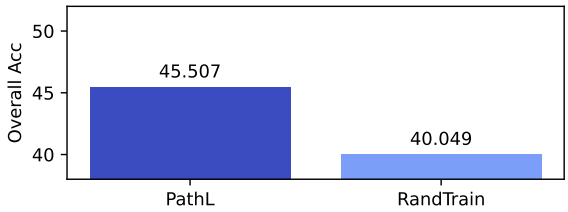


Figure 18: Comparison of PathPieceL and RandTrain, with n-gram initial vocab, and FirstSpace pre-tokenization, p=9.77e-4

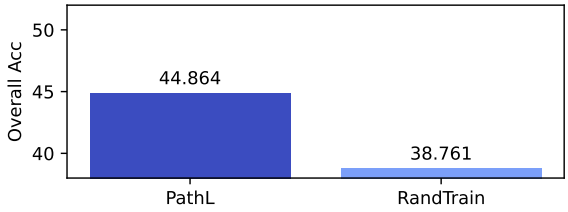


Figure 19: Comparison of PathPieceL and RandTrain, with n-gram initial vocab, and FirstSpaceDigits pre-tokenization, p=0.00977

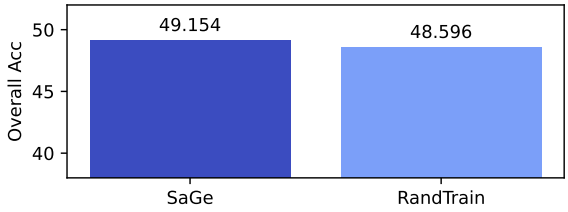


Figure 20: Comparison of SaGe and RandTrain, with BPE initial vocab, and FirstSpace pre-tokenization, p=0.0645

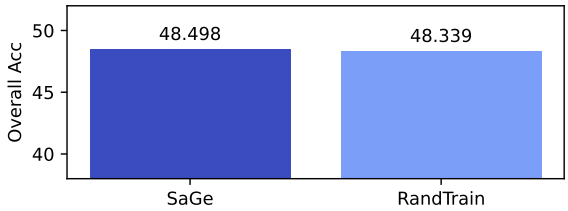


Figure 21: Comparison of SaGe and RandTrain, with n-gram initial vocab, and FirstSpace pre-tokenization, p=0.688

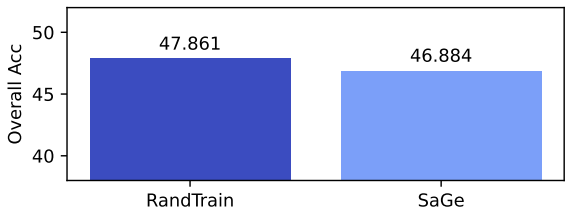


Figure 22: Comparison of RandTrain and SaGe, with n-gram initial vocab, and FirstSpaceDigit pre-tokenization, p=0.15

Vocab Constr	Init Voc	Pre-tok	Segment	Avg	arc_easy	copa	mktg	mathqa	piqa
BPE		FirstSpace	Merge	48.8	51.2	69.0	32.9	**23.9**	66.3
		FirstSpace	Greedy	48.3	51.9	66.0	32.9	23.7	65.6
		FirstSpace	PathPieceL	45.6	45.6	61.0	29.9	23.0	60.5
Unigram		FirstSpace	Likelihood	49.2	50.7	73.0	30.8	23.1	66.3
		FirstSpace	Greedy	47.9	50.3	68.0	31.2	23.1	65.2
		FirstSpace	PathPieceL	43.6	41.2	57.0	31.6	22.0	60.6
WordPiece		FirstSpace	Greedy	48.5	**52.5**	64.0	32.5	**23.9**	65.6
SaGe	BPE	FirstSpace	Greedy	47.9	49.7	67.0	26.5	23.2	65.9
	n-gram	FirstSpDigit	Greedy	48.4	50.3	71.0	29.5	22.0	65.1
	n-gram	FirstSpace	Greedy	47.5	48.8	64.0	29.5	23.0	**66.6**
	Unigram	FirstSpace	Greedy	48.4	52.0	**74.0**	27.8	22.7	65.7
PathPieceL	BPE	FirstSpace	PathPieceL	**49.3**	50.8	68.0	**34.2**	23.0	66.4
	n-gram	FirstSpace	PathPieceL	44.8	42.3	61.0	27.4	23.0	61.2
	n-gram	FirstSpDigit	PathPieceL	44.6	42.3	62.0	31.2	22.8	61.2
	Unigram	FirstSpace	PathPieceL	46.9	50.4	64.0	24.8	23.5	66.2
PathPieceR	n-gram	FirstSpDigit	PathPieceR	45.3	46.9	67.0	26.9	22.4	59.9
	n-gram	None	PathPieceR	43.5	42.5	65.0	26.1	22.8	61.7
	n-gram	SpaceDigit	PathPieceR	47.5	48.6	68.0	32.9	23.3	65.0
Random				32.0	25.0	50.0	25.0	20.0	50.00

Table 4: 350M parameter model, 32,768 token vocabulary, accuracy (%) on average and initial 5 tasks

Vocab Constr	Init Voc	Pre-tok	Segment	qa4mre	race	sciq	sociology	wsc273
BPE		FirstSpace	Merge	29.6	29.2	87.3	30.9	67.8
		FirstSpace	Greedy	27.5	30.7	88.0	30.9	66.3
		FirstSpace	PathPieceL	28.2	29.0	83.8	28.4	66.3
Unigram		FirstSpace	Likelihood	31.0	30.2	86.4	31.8	**68.5**
		FirstSpace	Greedy	28.9	30.6	86.9	31.8	62.6
		FirstSpace	PathPieceL	29.9	27.5	74.6	26.4	65.6
WordPiece		FirstSpace	Greedy	**32.0**	30.7	88.5	27.9	67.4
SaGe	BPE	FirstSpace	Greedy	31.7	30.2	**89.0**	28.4	67.8
	n-gram	FirstSpDigit	Greedy	31.0	30.3	86.6	32.3	66.0
	n-gram	FirstSpace	Greedy	30.0	31.0	87.8	25.9	**68.5**
	Unigram	FirstSpace	Greedy	29.6	28.9	88.2	32.3	63.0
PathPieceL	BPE	FirstSpace	PathPieceL	28.5	**31.1**	88.8	**35.3**	67.0
	n-gram	FirstSpace	PathPieceL	30.3	27.3	80.0	32.8	62.6
	n-gram	FirstSpDigit	PathPieceL	27.8	25.5	79.2	31.3	62.6
	Unigram	FirstSpace	PathPieceL	29.6	30.6	87.6	24.4	68.1
PathPieceR	n-gram	FirstSpDigit	PathPieceR	28.5	29.4	78.6	28.9	64.5
	n-gram	None	PathPieceR	27.1	27.0	77.7	28.9	56.0
	n-gram	SpaceDigit	PathPieceR	25.0	29.4	85.7	32.3	64.8
Random				25.0	25.0	25.0	25.0	50.0

Table 5: 350M parameter model, 32,768 token vocabulary, accuracy (%) on remaining 5 tasks

Vocab Constr	Init Voc	Pre-tok	Segment	Avg	arc_easy	copa	mktg	mathqa	piqa
BPE		FirstSpace	Merge	**50.0**	**52.7**	70.0	31.6	24.3	66.9
		FirstSpace	Greedy	49.1	52.3	66.0	27.4	22.9	**66.9**
		FirstSpace	PathPieceL	46.7	48.0	58.0	27.4	23.4	62.1
Unigram		FirstSpace	Likelihood	49.1	51.4	**71.0**	32.1	23.4	66.1
		FirstSpace	Greedy	48.5	49.9	64.0	30.3	23.3	65.7
Unigram		FirstSpace	PathPieceL	43.1	40.5	56.0	28.6	23.0	60.3
WordPiece		FirstSpace	Greedy	49.1	52.3	70.0	28.6	23.7	66.5
SaGe	BPE	FirstSpace	Greedy	49.2	50.8	70.0	29.9	23.2	66.4
	n-gram	FirstSpDigit	Greedy	46.9	48.4	67.0	30.3	22.6	64.0
	n-gram	FirstSpace	Greedy	48.5	49.8	68.0	**32.9**	22.8	65.4
	Unigram	FirstSpace	Greedy	46.9	51.7	65.0	28.6	23.9	65.2
PathPieceL	BPE	FirstSpace	PathPieceL	49.4	52.1	**71.0**	29.9	23.9	66.9
	n-gram	FirstSpace	PathPieceL	45.5	42.6	63.0	30.3	22.7	60.9
	n-gram	FirstSpDigit	PathPieceL	44.9	44.0	60.0	29.9	22.6	60.8
	Unigram	FirstSpace	PathPieceL	48.5	51.7	**71.0**	31.2	24.2	66.2
PathPieceR	n-gram	FirstSpDigit	PathPieceR	45.8	47.5	63.0	28.2	22.4	60.7
	n-gram	None	PathPieceR	44.0	41.2	66.0	26.5	21.6	62.4
	n-gram	SpaceDigit	PathPieceR	45.4	46.3	64.0	32.1	22.7	60.0
RandTrain	BPE	FirstSpace	Greedy	48.6	50.5	70.0	29.5	23.4	65.8
	n-gram	FirstSpDigit	Greedy	47.9	50.0	63.0	29.5	23.3	65.3
	n-gram	FirstSpace	Greedy	48.3	50.3	70.0	28.2	**24.3**	65.8
	n-gram	None	Greedy	42.2	41.3	55.0	27.4	21.7	63.2
	BPE	FirstSpace	PathPieceL	46.5	45.8	65.0	30.8	23.3	62.8
	n-gram	FirstSpDigit	PathPieceL	38.8	31.2	48.0	27.8	22.6	54.7
	n-gram	FirstSpace	PathPieceL	40.0	30.7	55.0	26.5	20.8	55.4
	n-gram	None	PathPieceL	36.8	27.7	56.0	28.6	22.8	54.5
random				32.0	25.0	50.0	25.0	20.0	50.0

Table 6: 350M parameter model, 40,960 token vocabulary, accuracy (%) on average and initial 5 tasks

Vocab Constr	Init Voc	Pre-tok	Segment	qa4mre	race	sciq	sociology	wsc273
BPE		FirstSpace	Merge	32.4	30.1	87.7	35.3	69.2
		FirstSpace	Greedy	31.7	**30.9**	88.3	**35.8**	68.9
		FirstSpace	PathPieceL	30.3	30.2	83.8	35.3	68.1
Unigram		FirstSpace	Likelihood	29.6	30.8	86.4	32.8	67.8
		FirstSpace	Greedy	32.4	29.6	86.7	32.8	**70.3**
		FirstSpace	PathPieceL	30.3	27.4	75.0	27.4	62.3
WordPiece		FirstSpace	Greedy	31.0	30.3	87.7	32.8	68.1
SaGe	BPE	FirstSpace	Greedy	28.9	30.2	**89.5**	34.8	67.8
	n-gram	FirstSpDigit	Greedy	30.6	28.1	85.8	32.3	59.7
	n-gram	FirstSpace	Greedy	29.2	30.0	88.4	33.3	65.2
	Unigram	FirstSpace	Greedy	26.8	29.1	86.9	31.3	60.1
PathPieceL	BPE	FirstSpace	PathPieceL	31.0	29.6	87.3	34.3	67.8
	n-gram	FirstSpace	PathPieceL	29.9	27.9	81.0	34.8	61.9
	n-gram	FirstSpDigit	PathPieceL	27.5	28.2	80.7	30.9	64.1
	Unigram	FirstSpace	PathPieceL	31.3	29.7	86.3	29.9	63.7
PathPieceR	n-gram	FirstSpDigit	PathPieceR	29.9	30.8	82.1	27.4	66.3
	n-gram	None	PathPieceR	23.6	28.3	73.8	**35.8**	60.4
	n-gram	SpaceDigit	PathPieceR	27.5	28.7	78.2	31.3	63.0
RandTrain	BPE	FirstSpace	Greedy	32.0	29.6	86.9	30.9	67.4
	n-gram	FirstSpDigit	Greedy	30.6	30.0	87.5	31.3	68.1
	n-gram	FirstSpace	Greedy	29.9	29.7	85.3	32.8	67.0
	n-gram	None	Greedy	28.2	27.8	75.9	26.4	55.0
	BPE	FirstSpace	PathPieceL	**32.8**	28.5	80.3	30.9	64.5
	n-gram	FirstSpDigit	PathPieceL	31.3	24.2	62.1	30.4	55.3
	n-gram	FirstSpace	PathPieceL	28.9	23.6	66.8	33.8	59.0
	n-gram	None	PathPieceL	21.5	24.9	51.8	28.9	51.7
random				25.0	25.0	25.0	25.0	50.0

Table 7: 350M parameter model, 40,960 token vocabulary, accuracy (%) on remaining 5 tasks

Vocab Constr	Init Voc	Pre-tok	Segment	Avg	arc_easy	copa	mktg	mathqa	piqa
BPE		FirstSpace	Merge	48.1	52.3	65.0	31.6	23.7	65.7
		FirstSpace	Greedy	**49.5**	**53.9**	**72.0**	31.6	24.2	**68.4**
		FirstSpace	PathPieceL	47.2	48.6	69.0	26.9	22.8	63.1
Unigram		FirstSpace	Likelihood	48.8	52.3	69.0	**35.0**	23.9	66.1
		FirstSpace	Greedy	48.6	51.6	68.0	32.1	24.4	65.7
		FirstSpace	PathPieceL	44.0	39.4	57.0	30.3	23.3	61.2
WordPiece		FirstSpace	Greedy	48.8	52.6	68.0	28.2	23.5	66.2
SaGe	BPE	FirstSpace	Greedy	48.8	51.9	71.0	29.9	22.6	65.5
	n-gram	FirstSpDigit	Greedy	47.2	46.6	67.0	31.2	22.7	63.4
	n-gram	FirstSpace	Greedy	48.0	49.7	66.0	31.6	21.6	65.7
	Unigram	FirstSpace	Greedy	47.8	49.7	68.0	29.9	23.5	64.6
PathPieceL	BPE	FirstSpace	PathPieceL	49.4	51.9	69.0	29.9	24.5	66.6
	n-gram	FirstSpace	PathPieceL	43.9	42.4	56.0	28.6	23.8	60.3
	n-gram	FirstSpDigit	PathPieceL	45.0	44.5	59.0	28.2	22.3	59.5
	Unigram	FirstSpace	PathPieceL	48.4	51.4	67.0	29.5	**24.7**	65.2
PathPieceR	n-gram	FirstSpDigit	PathPieceR	45.5	46.0	62.0	25.6	22.1	61.6
	n-gram	None	PathPieceR	42.2	42.6	64.0	22.2	22.4	60.9
	n-gram	SpaceDigit	PathPieceR	47.3	48.7	68.0	34.2	21.9	65.1
random				32.0	25.0	50.0	25.0	20.0	50.0

Table 8: 350M parameter model, 49,152 token vocabulary, accuracy (%) on average and initial 5 tasks

Vocab Constr	Init Voc	Pre-tok	Segment	qa4mre	race	sciq	sociology	wsc273
BPE		FirstSpace	Merge	28.9	31.0	87.3	28.9	67.0
		FirstSpace	Greedy	29.6	31.2	88.4	29.4	66.3
		FirstSpace	PathPieceL	31.0	30.7	85.4	31.8	63.0
Unigram		FirstSpace	Likelihood	27.5	30.3	**89.1**	28.9	65.9
		FirstSpace	Greedy	32.4	29.5	86.7	32.3	63.7
		FirstSpace	PathPieceL	**33.1**	26.0	74.5	27.9	67.0
WordPiece		FirstSpace	Greedy	29.2	31.1	88.0	34.3	66.7
SaGe	BPE	FirstSpace	Greedy	29.6	31.2	87.5	32.3	65.9
	n-gram	FirstSpDigit	Greedy	29.2	28.8	86.4	34.3	61.9
	n-gram	FirstSpace	Greedy	28.8	30.2	87.5	33.8	64.5
	Unigram	FirstSpace	Greedy	28.9	**31.4**	87.0	29.9	65.6
PathPieceL	BPE	FirstSpace	PathPieceL	31.0	**31.4**	87.5	31.3	**70.7**
	n-gram	FirstSpace	PathPieceL	27.5	26.7	80.8	32.3	60.8
	n-gram	FirstSpDigit	PathPieceL	28.9	30.0	80.6	**35.8**	61.2
	Unigram	FirstSpace	PathPieceL	29.2	30.5	88.5	32.8	65.6
PathPieceR	n-gram	FirstSpDigit	PathPieceR	29.6	29.5	82.8	30.9	64.5
	n-gram	None	PathPieceR	25.7	27.5	72.5	27.4	57.1
	n-gram	SpaceDigit	PathPieceR	27.5	28.7	84.0	28.9	66.3
Random				25.0	25.0	25.0	25.0	50.0

Table 9: 350M parameter model, 49,152 token vocabulary, accuracy (%) on remaining 5 tasks

Rank	Vocab Constr	Init Voc	Pre-tok	Segment	Overall avg	32,768 avg	40,960 avg	49,152 avg
1	PathPieceL	BPE	FirstSpace	PathPieceL	**49.4**	**49.3**	49.4	49.4
2	Unigram		FirstSpace	Likelihood	49.0	49.2	49.1	48.8
3	BPE		FirstSpace	Merge	49.0	48.8	**50.0**	48.1
4	BPE		FirstSpace	Greedy	49.0	48.3	49.1	**49.5**
5	WordPiece		FirstSpace	Greedy	48.8	48.5	49.1	48.8
6	SaGe	BPE	FirstSpace	Greedy	48.6	47.9	49.2	48.8
7	Unigram		FirstSpace	Greedy	48.3	47.9	48.5	48.6
8	SaGe	n-gram	FirstSpace	Greedy	48.0	47.5	48.5	48.0
9	PathPieceL	Unigram	FirstSpace	PathPieceL	48.0	46.9	48.5	48.4
10	SaGe	Unigram	FirstSpace	Greedy	47.7	48.4	46.9	47.8
11	SaGe	n-gram	FirstSpDigit	Greedy	47.5	48.4	46.9	47.2
12	PathPieceR	n-gram	SpaceDigit	PathPieceR	46.7	47.5	45.4	47.3
13	BPE		FirstSpace	PathPieceL	46.5	45.6	46.7	47.2
14	PathPieceR	n-gram	FirstSpDigit	PathPieceR	45.5	45.3	45.8	45.5
15	PathPieceL	n-gram	FirstSpDigit	PathPieceL	44.8	44.6	44.9	45.0
16	PathPieceL	n-gram	FirstSpace	PathPieceL	44.7	44.8	45.5	43.9
17	Unigram		FirstSpace	PathPieceL	43.6	43.6	43.1	44.0
18	PathPieceR	n-gram	None	PathPieceR	43.2	43.5	44.0	42.2
	Random				32.0	32.0	32.0	32.0

Table 10: Summary of 350M parameter model downstream accuracy (%), sorted by rank

Rank	Vocab Size	Avg Acc	CTC	Eff α=1.5	Eff α=2	Eff α=2.5	Eff α=3	Eff α=3.5
1	32,768	49.3	1.48	0.604	0.516	0.469	0.441	0.422
1	40,960	49.4	1.46	0.589	0.503	0.457	0.429	0.411
1	49,152	49.4	1.44	0.578	0.492	0.448	0.420	0.402
2	32,768	49.2	1.79	0.461	0.371	0.324	0.295	0.277
2	40,960	49.1	1.77	0.451	0.362	0.316	0.289	0.271
2	49,152	48.8	1.76	0.444	0.356	0.311	0.284	0.266
3	32,768	48.8	1.52	0.594	0.505	0.459	0.431	0.414
3	40,960	50.0	1.49	0.579	0.491	0.446	0.420	0.403
3	49,152	48.1	1.47	0.567	0.481	0.437	0.411	0.394
4	32,768	48.3	1.50	0.605	0.517	0.471	0.442	0.423
4	40,960	49.1	1.48	0.590	0.504	0.458	0.430	0.412
4	49,152	49.5	1.46	0.579	0.494	0.449	0.421	0.403
5	32,768	48.5	1.54	0.598	0.507	0.461	0.433	0.415
5	40,960	49.1	1.51	0.583	0.494	0.448	0.421	0.404
5	49,152	48.8	1.49	0.571	0.483	0.439	0.412	0.396
6	32,768	47.9	1.78	0.545	0.466	0.422	0.396	0.378
6	40,960	49.2	1.76	0.533	0.455	0.413	0.387	0.369
6	49,152	48.7	1.75	0.523	0.447	0.405	0.379	0.362
7	32,768	47.9	1.81	0.510	0.431	0.387	0.359	0.340
7	40,960	48.5	1.79	0.500	0.423	0.381	0.354	0.335
7	49,152	48.6	1.77	0.493	0.416	0.375	0.348	0.330
8	32,768	47.5	1.63	0.629	0.536	0.482	0.447	0.424
8	40,960	48.5	1.62	0.615	0.524	0.470	0.437	0.415
8	49,152	48.0	1.62	0.605	0.515	0.462	0.429	0.407
9	32,768	46.9	1.74	0.508	0.419	0.372	0.343	0.323
9	40,960	48.5	1.72	0.491	0.403	0.356	0.328	0.309
9	49,152	48.4	1.72	0.477	0.389	0.343	0.315	0.296
10	32,768	48.4	2.02	0.485	0.409	0.366	0.339	0.320
10	40,960	46.9	2.01	0.474	0.401	0.358	0.331	0.313
10	49,152	47.8	2.01	0.466	0.393	0.352	0.325	0.307
11	32,768	48.4	1.77	0.587	0.512	0.470	0.443	0.425
11	40,960	46.9	1.76	0.575	0.501	0.460	0.433	0.415
11	49,152	47.2	1.76	0.565	0.492	0.452	0.426	0.408
12	32,768	47.5	2.33	0.236	0.164	0.138	0.124	0.116
12	40,960	45.4	2.30	0.228	0.159	0.133	0.120	0.112
12	49,152	47.3	2.29	0.223	0.155	0.130	0.117	0.109
13	32,768	45.6	1.50	0.606	0.518	0.470	0.442	0.423
13	40,960	46.7	1.47	0.591	0.504	0.458	0.430	0.412
13	49,152	47.2	1.45	0.579	0.494	0.449	0.421	0.403
14	32,768	45.3	1.46	0.616	0.532	0.490	0.465	0.448
14	40,960	45.8	1.43	0.602	0.519	0.478	0.453	0.437
14	49,152	45.5	1.42	0.591	0.508	0.468	0.444	0.428
15	32,768	44.6	1.47	0.620	0.533	0.490	0.464	0.447
15	40,960	44.9	1.44	0.605	0.520	0.478	0.453	0.436
15	49,152	45.0	1.42	0.594	0.509	0.468	0.443	0.427
16	32,768	44.8	1.36	0.677	0.571	0.514	0.480	0.457
16	40,960	45.5	1.33	0.662	0.556	0.500	0.466	0.444
16	49,152	43.9	1.31	0.650	0.544	0.489	0.456	0.435
17	32,768	43.6	1.77	0.471	0.380	0.333	0.304	0.285
17	40,960	43.1	1.75	0.462	0.372	0.326	0.298	0.280
17	49,152	44.0	1.74	0.455	0.366	0.320	0.293	0.275
18	32,768	43.5	1.29	0.747	0.617	0.549	0.511	0.486
18	40,960	44.0	1.26	0.736	0.603	0.535	0.497	0.474
18	49,152	42.2	1.25	0.728	0.591	0.524	0.487	0.464

Table 11: Average Accuracy (%) vs. Corpus Token Count (CTC, in billions) by vocabulary size, for Figure 3. Also includes the corresponding Rényi efficiency (Zouhar et al., 2023a) for various orders α.

Vocab Constr	Init Voc	Pre-tok	Segment	Avg	arc_easy	copa	mktg	mathqa	piqa
BPE		FirstSpace	Merge	**53.1**	**62.0**	**77.0**	**32.1**	25.0	71.1
Unigram		FirstSpace	Likelihood	52.4	60.6	71.0	30.3	**25.2**	71.0
SaGe	BPE	FirstSpace	Greedy	52.2	62.0	72.0	27.4	24.5	**71.6**
	n-gram	FirstSpDigit	Greedy	50.7	60.3	71.0	28.6	22.8	69.4
PathPieceL	BPE	FirstSpace	PathPieceL	49.2	57.4	66.0	27.8	24.3	65.9
	n-gram	FirstSpDigit	PathPieceL	47.6	49.7	67.0	24.8	23.4	63.2
	n-gram	SpaceDigit	PathPieceL	46.3	51.1	59.0	28.6	23.3	63.8
Random				32.0	25.0	50.0	25.0	20.0	50.0

Table 12: 1.3B parameter model, 40,960 token vocabulary, accuracy (%) on average and initial 5 tasks

Vocab Constr	Init Voc	Pre-tok	Segment	qa4mre	race	sciq	sociology	wsc273
BPE		FirstSpace	Merge	32.4	**34.9**	**93.0**	26.4	**76.9**
Unigram		FirstSpace	Likelihood	**37.7**	33.0	91.8	28.9	74.4
SaGe	BPE	FirstSpace	Greedy	34.9	34.8	92.5	25.9	76.2
	n-gram	FirstSpDigit	Greedy	29.9	32.9	91.5	**29.4**	71.1
PathPieceL	BPE	FirstSpace	PathPieceL	31.0	33.3	89.4	26.4	70.7
	n-gram	FirstSpDigit	PathPieceL	31.0	31.6	86.1	**29.4**	70.0
	n-gram	SpaceDigit	PathPieceL	28.9	31.3	87.1	22.4	67.0
Random				25.0	25.0	25.0	25.0	50.0

Table 13: 1.3B parameter model, 40,960 token vocabulary, accuracy (%) on remaining 5 tasks

Voc Con	Init V	Pre-tok	Seg	350M avg	350M rnk	1.3B avg	1.3B rnk	2.4B avg	2.4B rnk
BPE		FirSp	Merge	50.0	1	53.1	1	54.2	3
PathPL	BPE	FirSp	PathPL	49.4	3	49.2	5	52.7	4
PathPL	n-gram	FirSpD	PathPL	44.9	6	47.6	6		
SaGe	BPE	FirSp	Greedy	49.2	2	52.2	3	55.0	1
SaGe	n-gram	FirSpD	Greedy	46.9	5	50.7	4		
Unigram		FirSp	Likeli	49.1	4	52.4	2	54.7	2

Table 14: Downstream accuracy (%) of 10 tasks with vocab size 40,960, for various model sizes